← SecureGate AI Security Gate

# CICD file updated #3

## Workflow file

### Workflow file for this run

.github/workflows/securegate.yml at ccccf36

```yaml
name: SecureGate AI Security Gate

on:
  push:
    branches:
      - "main"
  workflow_dispatch:

jobs:
  securegate-security-check:
    runs-on: ubuntu-latest

    env:
      SECUREGATE_URL: ${{ secrets.SECUREGATE_URL }}
      SECUREGATE_API_KEY: ${{ secrets.SECUREGATE_API_KEY }}
      SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
      SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
      SONAR_PROJECT_KEY: ${{ secrets.SONAR_PROJECT_KEY }}
      SONAR_ORGANIZATION: ${{ secrets.SONAR_ORGANIZATION }}
      DOCKERHUB_USERNAME: ${{ secrets.DOCKERHUB_USERNAME }}
      DOCKERHUB_TOKEN: ${{ secrets.DOCKERHUB_TOKEN }}
      PROJECT_NAME: ${{ github.event.repository.name }}
      REPOSITORY_URL: ${{ github.server_url }}/${{ github.repository }}
      BRANCH_NAME: ${{ github.ref_name }}
      PIPELINE_RUN_ID: ${{ github.run_id }}
      REPOSITORY_OWNER: ${{ github.repository_owner }}

    steps:
      - name: Checkout repository
```

```
30          uses: actions/checkout@93cb6efe18208431cddfb8368fd83d5badbf9bfd
31
32        - name: Detect current commit SHA
33          run: |
34            CURRENT_COMMIT_SHA=$(git rev-parse HEAD)
35            echo "COMMIT_SHA=$CURRENT_COMMIT_SHA" >> "$GITHUB_ENV"
36            echo "Current commit SHA: $CURRENT_COMMIT_SHA"
37
38        - name: Setup Node.js 24
39          uses: actions/setup-node@a0853c24544627f65ddf259abe73b1d18a591444
40          with:
41            node-version: "24"
42
43        - name: Validate SecureGate AI URL
44          run: |
45            if [ -z "$SECUREGATE_URL" ]; then
46              echo "SECUREGATE_URL GitHub secret is missing."
47              exit 1
48            fi
49
50            if echo "$SECUREGATE_URL" | grep -Eq '\[|\]|\(|\)| '; then
51              echo "SECUREGATE_URL is malformed."
52              echo "Save it as a plain URL only, for example: https://citric-pliable-baggie.ngrok-free.de
53              echo "Do not use Markdown format like [https://...](https://...)."
54              exit 1
55            fi
56
57            case "$SECUREGATE_URL" in
58              http://*|https://*) ;;
59              *)
60                echo "SECUREGATE_URL must start with http:// or https://"
61                exit 1
62                ;;
63            esac
64
65        - name: Start SecureGate AI scan
66          id: securegate_start
67          run: |
68            if echo "$REPOSITORY_URL" | grep -Eq '\[|\]|\(|\)'; then
69              echo "REPOSITORY_URL is malformed. It must be a plain URL, not a Markdown link: $REPOSITORY
70              exit 1
71            fi
72
73            START_PAYLOAD=$(jq -nc \
74              --arg project_name "$PROJECT_NAME" \
75              --arg repository_url "$REPOSITORY_URL" \
76              --arg branch "$BRANCH_NAME" \
77              --arg commit_sha "$COMMIT_SHA" \
78              --arg pipeline_run_id "$PIPELINE_RUN_ID" \
```

```
            '{project_name:$project_name, repository_url:$repository_url, branch:$branch, commit_sha:$c

          echo "SecureGate start payload:"
          echo "$START_PAYLOAD" | jq .

          HTTP_STATUS=$(curl -sS -o securegate-start-response.json -w "%{http_code}" -X POST "${SECUREG
            -H "Authorization: Bearer $SECUREGATE_API_KEY" \
            -H "Content-Type: application/json" \
            --data "$START_PAYLOAD")

          echo "SecureGate start response status: $HTTP_STATUS"
          cat securegate-start-response.json
          if [ "$HTTP_STATUS" -lt 200 ] || [ "$HTTP_STATUS" -ge 300 ]; then
            echo "SecureGate AI rejected the pipeline start request. Check the validation details above
            exit 1
          fi

          RESPONSE=$(cat securegate-start-response.json)
          SCAN_ID=$(echo "$RESPONSE" | jq -r .scan_id)
          if [ -z "$SCAN_ID" ] || [ "$SCAN_ID" = "null" ]; then
            echo "SecureGate AI did not return a scan_id"
            exit 1
          fi
          echo "scan_id=$SCAN_ID" >> "$GITHUB_OUTPUT"

      - name: Run GitLeaks secret scan
        run: |
          docker run --rm \
            -v "$PWD:/repo" \
            zricethezav/gitleaks:latest detect \
            --source=/repo \
            --report-format=json \
            --report-path=/repo/gitleaks-report.json \
            --no-git || true

          test -f gitleaks-report.json || echo "[]" > gitleaks-report.json

      - name: Run Semgrep SAST scan
        run: |
          docker run --rm -v "$PWD:/src" returntocorp/semgrep:latest semgrep scan --config auto --json
          test -f semgrep-report.json || echo '{"results":[]}' > semgrep-report.json

      - name: Run Trivy filesystem scan
        uses: aquasecurity/trivy-action@df678dbccf677bc6937338d569ba0da18964a8cc
        with:
          scan-type: fs
          scan-ref: .
          format: json
          output: trivy-fs-report.json
```

```
          exit-code: 0

      - name: Ensure Trivy filesystem report exists
        run: |
          test -f trivy-fs-report.json || echo '{"Results":[]}' > trivy-fs-report.json

      - name: Run SonarQube scan
        if: env.SONAR_HOST_URL != '' && env.SONAR_TOKEN != '' && env.SONAR_PROJECT_KEY != '' && env.SON
        uses: SonarSource/sonarqube-scan-action@fd88b7d7ccbaefd23d8f36f73b59db7a3d246602 # nosemgrep: g
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
        with:
          args: >
            -Dsonar.projectKey=${{ secrets.SONAR_PROJECT_KEY }}
            -Dsonar.organization=${{ secrets.SONAR_ORGANIZATION }}
            -Dsonar.sources=.
            -Dsonar.exclusions=.github/workflows/securegate.yml,.github/workflows/securegate-ai.yml,Sec

      - name: Fetch SonarQube quality gate and issues
        if: env.SONAR_HOST_URL != '' && env.SONAR_TOKEN != '' && env.SONAR_PROJECT_KEY != '' && env.SON
        run: |
          curl -sS \
            -H "Authorization: Bearer $SONAR_TOKEN" \
            "$SONAR_HOST_URL/api/qualitygates/project_status?projectKey=$SONAR_PROJECT_KEY&organization
            -o sonar-gate.json

          curl -sS \
            -H "Authorization: Bearer $SONAR_TOKEN" \
            "$SONAR_HOST_URL/api/issues/search?componentKeys=$SONAR_PROJECT_KEY&organization=$SONAR_ORG
            -o sonar-issues.json

          jq -n \
            --slurpfile gate sonar-gate.json \
            --slurpfile issues sonar-issues.json \
            '{projectStatus: $gate[0].projectStatus, issues: ($issues[0].issues // [])}' \
            > sonarqube-report.json

      - name: Create SonarQube placeholder report
        if: env.SONAR_HOST_URL == '' || env.SONAR_TOKEN == '' || env.SONAR_PROJECT_KEY == '' || env.SON
        run: |
          cat > sonarqube-report.json <<EOF
          {
            "projectStatus": {
              "status": "UNKNOWN"
            },
            "issues": []
          }
          EOF
```

```
177
178       - name: Detect and run unit tests
179         run: |
180           STATUS="SKIPPED"
181           FRAMEWORK="none"
182           REASON="No supported unit test configuration found"
183           TOTAL=0
184           FAILED=0
185
186           if [ -f package.json ] && jq -e '.scripts.test and .scripts.test != ""' package.json >/dev/nu
187             FRAMEWORK="node"
188             if npm test; then STATUS="PASSED"; REASON="Node test script passed"; else STATUS="FAILED";
189           elif [ -d tests ] || [ -f pytest.ini ] || grep -q "pytest" pyproject.toml setup.cfg 2>/dev/nu
190             FRAMEWORK="pytest"
191             python -m pip install --upgrade pip >/dev/null 2>&1 || true
192             if [ -f requirements.txt ]; then python -m pip install -r requirements.txt >/dev/null 2>&1
193             if [ -f requirements-dev.txt ]; then python -m pip install -r requirements-dev.txt >/dev/nu
194             python -m pip install pytest >/dev/null 2>&1 || true
195             if python -m pytest; then STATUS="PASSED"; REASON="pytest passed"; else STATUS="FAILED"; RE
196           fi
197
198           jq -n --arg status "$STATUS" --arg framework "$FRAMEWORK" --arg reason "$REASON" --argjson to
199             '{status:$status, framework:$framework, reason:$reason, total:$total, failed:$failed}' > un
200
201       - name: Prepare Dockerfile when missing
202         id: dockerfile
203         run: |
204           if [ -f Dockerfile ]; then
205             echo "Using repository Dockerfile."
206             echo "path=Dockerfile" >> "$GITHUB_OUTPUT"
207             exit 0
208           fi
209
210           echo "No Dockerfile found. Creating SecureGate.Dockerfile for image scanning."
211           echo "path=SecureGate.Dockerfile" >> "$GITHUB_OUTPUT"
212
213           if [ -f package.json ]; then
214             cat > SecureGate.Dockerfile <<'EOF'
215           FROM node:24-alpine
216           WORKDIR /app
217           COPY package*.json ./
218           RUN npm install
219           COPY . .
220           CMD ["npm", "start"]
221           EOF
222           elif [ -f requirements.txt ]; then
223             cat > SecureGate.Dockerfile <<'EOF'
224           FROM python:3.12-alpine
225           WORKDIR /app
```

```
          COPY requirements.txt ./
          RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.tx
          COPY . .
          CMD ["python", "app.py"]
          EOF
          else
            cat > SecureGate.Dockerfile <<'EOF'
          FROM alpine:3.20
          WORKDIR /app
          COPY . .
          CMD ["sh", "-c", "ls -la /app && sleep 30"]
          EOF
          fi

      - name: Build Docker image for image vulnerability scan
        id: image_build
        run: |
          IMAGE_NAMESPACE="${DOCKERHUB_USERNAME:-$REPOSITORY_OWNER}"
          IMAGE_REPOSITORY="$PROJECT_NAME"
          IMAGE_REPO=$(echo "$IMAGE_NAMESPACE/$IMAGE_REPOSITORY" | tr '[:upper:]' '[:lower:]')
          SHORT_COMMIT_SHA="${COMMIT_SHA:0:8}"
          IMAGE_SHA_REF="$IMAGE_REPO:$SHORT_COMMIT_SHA"
          IMAGE_LATEST_REF="$IMAGE_REPO:latest"

          echo "SecureGate AI Docker image path: $IMAGE_SHA_REF"
          echo "Docker Hub repository will be created automatically on first push when credentials allo

          docker build -f "${{ steps.dockerfile.outputs.path }}" -t "$IMAGE_SHA_REF" .
          docker tag "$IMAGE_SHA_REF" "$IMAGE_LATEST_REF"

          echo "image_ref=$IMAGE_SHA_REF" >> "$GITHUB_OUTPUT"
          echo "latest_ref=$IMAGE_LATEST_REF" >> "$GITHUB_OUTPUT"
          echo "image_repo=$IMAGE_REPO" >> "$GITHUB_OUTPUT"

      - name: Check Docker Hub image tag freshness
        id: image_freshness
        if: env.DOCKERHUB_USERNAME != '' && env.DOCKERHUB_TOKEN != ''
        env:
          IMAGE_REF: ${{ steps.image_build.outputs.image_ref }}
        run: |
          if docker manifest inspect "$IMAGE_REF" >/dev/null 2>&1; then
            echo "changed=false" >> "$GITHUB_OUTPUT"
            echo "Image already exists on Docker Hub for this short commit tag. Push will be skipped."
          else
            echo "changed=true" >> "$GITHUB_OUTPUT"
            echo "No existing Docker Hub image found for this short commit tag. Push is needed."
          fi

      - name: Run Trivy image scan
```

```
275         uses: aquasecurity/trivy-action@df678dbccf677bc6937338d569ba0da18964a8cc
276         with:
277           image-ref: ${{ steps.image_build.outputs.image_ref }}
278           format: json
279           output: trivy-image-report.json
280           exit-code: 0
281
282       - name: Ensure Trivy image report exists
283         run: |
284           test -f trivy-image-report.json || echo '{"Results":[]}' > trivy-image-report.json
285
286       - name: Upload GitLeaks report
287         env:
288           SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
289         run: |
290           curl -fsS -X POST "${SECUREGATE_URL%/}/api/v1/scans/$SCAN_ID/gitleaks" \
291             -H "Authorization: Bearer $SECUREGATE_API_KEY" \
292             -H "Content-Type: application/json" \
293             --data-binary @gitleaks-report.json
294
295       - name: Upload Semgrep report
296         env:
297           SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
298         run: |
299           curl -fsS -X POST "${SECUREGATE_URL%/}/api/v1/scans/$SCAN_ID/semgrep" \
300             -H "Authorization: Bearer $SECUREGATE_API_KEY" \
301             -H "Content-Type: application/json" \
302             --data-binary @semgrep-report.json
303
304       - name: Upload Trivy filesystem report
305         env:
306           SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
307         run: |
308           curl -fsS -X POST "${SECUREGATE_URL%/}/api/v1/scans/$SCAN_ID/trivy-fs" \
309             -H "Authorization: Bearer $SECUREGATE_API_KEY" \
310             -H "Content-Type: application/json" \
311             --data-binary @trivy-fs-report.json
312
313       - name: Upload Trivy image report
314         env:
315           SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
316         run: |
317           curl -fsS -X POST "${SECUREGATE_URL%/}/api/v1/scans/$SCAN_ID/trivy-image" \
318             -H "Authorization: Bearer $SECUREGATE_API_KEY" \
319             -H "Content-Type: application/json" \
320             --data-binary @trivy-image-report.json
321
322       - name: Upload SonarQube report
323         env:
```

```
          SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
        run: |
          curl -fsS -X POST "${SECUREGATE_URL%/}/api/v1/scans/$SCAN_ID/sonarqube" \
            -H "Authorization: Bearer $SECUREGATE_API_KEY" \
            -H "Content-Type: application/json" \
            --data-binary @sonarqube-report.json

      - name: Upload unit-test report
        env:
          SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
        run: |
          curl -fsS -X POST "${SECUREGATE_URL%/}/api/v1/scans/$SCAN_ID/unit-tests" \
            -H "Authorization: Bearer $SECUREGATE_API_KEY" \
            -H "Content-Type: application/json" \
            --data-binary @unit-tests-report.json

      - name: Check SecureGate AI deployment decision
        env:
          SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
        run: |
          RESPONSE=$(curl -fsS "${SECUREGATE_URL%/}/api/v1/scans/$SCAN_ID")
          echo "$RESPONSE"

          DECISION=$(echo "$RESPONSE" | jq -r .deployment_decision)
          SCORE=$(echo "$RESPONSE" | jq -r .security_score)
          SONAR_GATE=$(echo "$RESPONSE" | jq -r .sonar_quality_gate)
          FINDING_COUNTS=$(echo "$RESPONSE" | jq -c .finding_counts)
          TOOL_STATUS=$(echo "$RESPONSE" | jq -c .tool_status)

          echo "SecureGate AI Decision: $DECISION"
          echo "Security Score: $SCORE"
          echo "SonarQube Gate: $SONAR_GATE"
          echo "Finding Counts: $FINDING_COUNTS"
          echo "Tool Status: $TOOL_STATUS"

          if [ "$DECISION" != "APPROVED" ]; then
            echo "SecureGate AI rejected this build."
            if [ "$SONAR_GATE" != "OK" ] && [ "$SONAR_GATE" != "UNKNOWN" ]; then
              echo "Rejection reason: SonarQube quality gate is $SONAR_GATE. Check SonarCloud Quality G
            fi
            echo "Top findings causing/relevant to rejection:"
            echo "$RESPONSE" | jq -r '.findings[:10][] | "- [\(.tool)] \(.severity) \(.title) in \(.fil
            exit 1
          fi

          echo "SecureGate AI approved this build."
          echo "The code is safe to deploy the Docker image and push it to Docker Hub."
          echo "Repository, filesystem, image, secrets, and Sonar quality checks passed SecureGate AI p
```

```
373         - name: Push approved Docker image to Docker Hub when configured
374           if: env.DOCKERHUB_USERNAME != '' && env.DOCKERHUB_TOKEN != '' && steps.image_freshness.outputs.
375           env:
376             IMAGE_REF: ${{ steps.image_build.outputs.image_ref }}
377             LATEST_REF: ${{ steps.image_build.outputs.latest_ref }}
378           run: |
379             echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_USERNAME" --password-stdin
380             docker push "$IMAGE_REF"
381             docker push "$LATEST_REF"
382             echo "Docker image pushed to Docker Hub:"
383             echo "  $IMAGE_REF"
384             echo "  $LATEST_REF"
385
386         - name: Rescan pushed Docker image
387           if: env.DOCKERHUB_USERNAME != '' && env.DOCKERHUB_TOKEN != '' && steps.image_freshness.outputs.
388           uses: aquasecurity/trivy-action@df678dbccf677bc6937338d569ba0da18964a8cc
389           with:
390             image-ref: ${{ steps.image_build.outputs.image_ref }}
391             format: json
392             output: trivy-image-report.json
393             exit-code: 0
394
395         - name: Upload post-push image rescan to SecureGate AI
396           if: env.DOCKERHUB_USERNAME != '' && env.DOCKERHUB_TOKEN != '' && steps.image_freshness.outputs.
397           env:
398             SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
399           run: |
400             curl -fsS -X POST "${SECUREGATE_URL%/}/api/v1/scans/$SCAN_ID/trivy-image" \
401               -H "Authorization: Bearer $SECUREGATE_API_KEY" \
402               -H "Content-Type: application/json" \
403               --data-binary @trivy-image-report.json
404             echo "Pushed Docker image was rescanned and SecureGate AI dashboard was updated."
405
406         - name: Docker Hub image unchanged
407           if: env.DOCKERHUB_USERNAME != '' && env.DOCKERHUB_TOKEN != '' && steps.image_freshness.outputs.
408           run: |
409             echo "SecureGate AI approved this build, but Docker Hub push was skipped because this commit
410
411         - name: Docker Hub push not configured
412           if: env.DOCKERHUB_USERNAME == '' || env.DOCKERHUB_TOKEN == ''
413           run: |
414             echo "SecureGate AI approved this build, but Docker Hub push was skipped because DOCKERHUB_US
415
416         - name: Complete SecureGate AI pipeline
417           if: always() && steps.securegate_start.outputs.scan_id != ''
418           env:
419             SCAN_ID: ${{ steps.securegate_start.outputs.scan_id }}
420             JOB_STATUS: ${{ job.status }}
421           run: |
```

```
          STATUS="SUCCESS"

          if [ "$JOB_STATUS" != "success" ]; then
            STATUS="FAILED"
          fi

          curl -fsS -X POST "${SECUREGATE_URL%/}/api/v1/pipeline/complete" \
            -H "Authorization: Bearer $SECUREGATE_API_KEY" \
            -H "Content-Type: application/json" \
            -d "{
              \"scan_id\":\"$SCAN_ID\",
              \"status\":\"$STATUS\"
            }"
```